Point-in-Time Recovery (PITR) for Postgre 16

---

1. Executive Summary

Point-in-Time Recovery (PITR) is a critical disaster recovery capability in Postgre that allows database administrators to restore a database cluster to any specific moment in the past.

This document covers:

- Architecture of PITR
- Prerequisites and configuration
- Backup strategy
- Recovery procedures
- Best practices
- A complete hands-on lab scenario

---

2. Introduction

2.1 What is PITR

Point-in-Time Recovery is the ability to restore a Postgre database to any consistent state between:

- the moment a base backup was taken
- and any later moment for which WAL files are available

2.2 Why PITR matters

Scenario	Without PITR	With PITR
Accidental DROP TABLE	Full data loss	Recover to before drop
Bad DELETE or UPDATE	Partial data loss	Recover to before change
Application corruption	Restore last backup only	Recover to exact moment
Deployment failure	Manual rollback	Automated point recovery

2.3 How PITR works

PITR relies on two components:

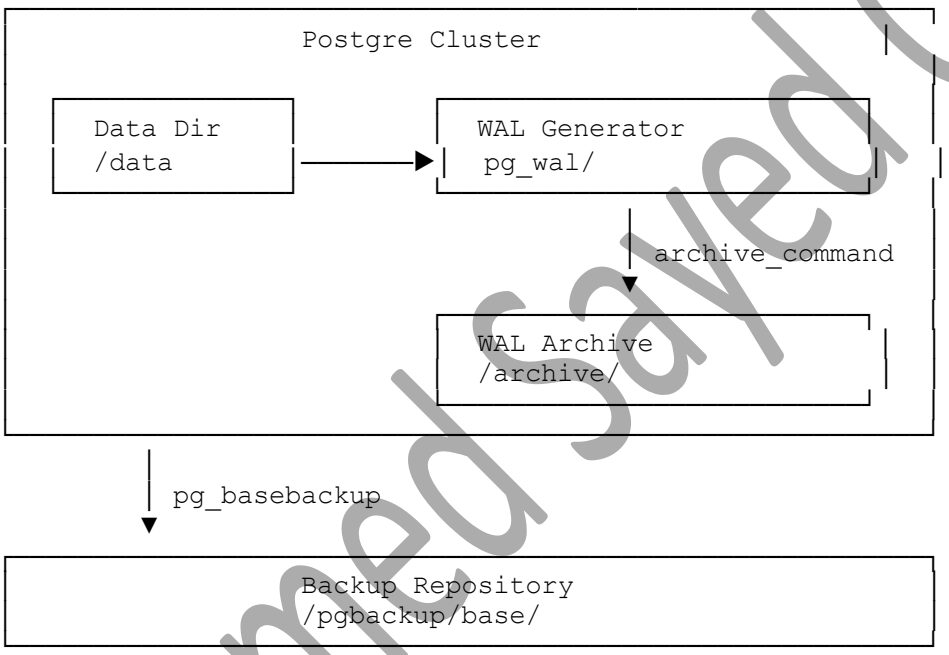
- **Base backup**: physical copy of the data directory
- **WAL files**: transaction log recording every change

Recovery replays WAL changes on top of the base backup until the target point is reached.

---

3. Architecture

3.1 PITR components



3.2 Recovery flow

```
    |                                                    (stop here)
    ▼
Promoted Primary
```

3.3 WAL archiving flow

```
Transaction committed
        |
        ▼
WAL segment written to pg_wal/
        |
        ▼
archive_command triggered
        |
        ▼
WAL file copied to archive directory
        |
        ▼
Archive confirmed
```

---

4. Prerequisites

4.1 Postgre configuration

The following parameters must be set in `postgre.conf`:

Parameter	Required Value	Purpose
wal_level	`replica` or higher	enable WAL for replication/archiving
archive_mode	`on`	enable WAL archiving
archive_command	valid copy command	define how WAL is archived
max_wal_senders	`>= 2`	allow replication connections
archive_timeout	`60` (recommended)	force WAL switch periodically

Example configuration

```
Vi postgre.conf

wal_level = replica
archive_mode = on
archive_command = 'test ! -f /var/lib/pg/16/archive/%f && cp %p
/var/lib/pg/16/archive/%f'
max_wal_senders = 10
archive_timeout = 60
```

```
listen_addresses = '*'
```

4.2 Authentication configuration

In `pg_hba.conf`:

```
conf


host    replication     replicator      127.0.0.1/32            md5
host    replication     replicator      10.110.95.0/24          md5
```

4.3 Backup user

```
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'StrongPassword';
```

4.4 Directory structure

Directory	Purpose
`/var/lib/pg/16/data`	Postgre data directory
`/var/lib/pg/16/archive`	WAL archive directory
`/pgbackup/base`	base backup storage

---

5. Backup Strategy

5.1 Recommended backup schedule

Frequency	Type	Purpose
Weekly	Full base backup	recovery starting point
Daily	WAL archive check	verify archive continuity
Continuous	WAL archiving	capture all changes
Monthly	Restore test	validate recovery capability

5.2 Retention policy

Item	Recommended Retention
Full base backups	4 weeks minimum
WAL archive files	cover all retained backups
Recovery logs	90 days

5.3 Base backup command

```
pg_basebackup \
  -h 127.0.0.1 \
  -p 5432 \
  -U replicator \
  -D /pgbackup/base/base_$(date +%F) \
  -F p \
  -X stream \
  -P \
  -c fast \
  -l "weekly_base_backup_$(date +%F)"
```

Options explained

Option	Meaning
`-h`	Postgre host
`-p`	Postgre port
`-U`	backup user
`-D`	destination directory
`-F p`	plain format
`-X stream`	stream WAL during backup
`-P`	show progress
`-c fast`	fast checkpoint
`-l`	backup label

---

6. Recovery Types

6.1 Recovery target options

Postgre 16 supports multiple recovery target types:

Target Type	Parameter	Example
Time	`recovery_target_time`	`'2026-06-09 14:35:22'`
Named point	`recovery_target_name`	`'before_release'`
LSN	`recovery_target_lsn`	`'0/1234567'`

Transaction ID	`recovery_target_xid`	`'12345'`
Latest	`recovery_target`	`'immediate'`

6.2 Recovery target actions

Action	Behavior
`promote`	promote to primary after recovery
`pause`	pause at target for inspection
`shutdown`	shutdown after reaching target

---

7. Complete PITR Procedure

7.1 Environment details

Item	Value
Postgre version	16
Data directory	`/var/lib/pg/16/data`
Archive directory	`/var/lib/pg/16/archive`
Backup directory	`/pgbackup/base/base_01`
Recovery target time	`2026-06-09 10:16:40`

7.2 Phase 1 — Verify archiving status

```
sudo -u postgres p -d postgres


SHOW archive_mode;
SHOW wal_level;
SHOW archive_command;

SELECT
    archived_count,
    last_archived_wal,
    last_archived_time,
    failed_count,
    last_failed_wal,
    last_failed_time
FROM pg_stat_archiver;
```

Expected output from your environment:

```
 archive_mode | on
 wal_level    | replica
 archive_command | cp %p /var/lib/pg/16/archive/%f

 archived_count | last_archived_wal        | last_archived_time
----------------+--------------------------+-------------------------------
            581 | 000000010000000200000042 | 2026-06-09 10:16:01.816472-04
```

7.3 Phase 2 — Take base backup

```
mkdir -p /pgbackup/base

export PGPASSWORD='ReplPass123'

pg_basebackup \
  -h 127.0.0.1 \
  -p 5432 \
  -U replicator \
  -D /pgbackup/base/base_01 \
  -F p \
  -X stream \
  -P \
  -c fast \
  -l "pitr_lab_base_backup"

unset PGPASSWORD
```

Verify:

```
ls -lh /pgbackup/base/base_01/
du -sh /pgbackup/base/base_01/
```

7.4 Phase 3 — Create test data

```
sudo -u postgres p -d postgres
```

```
-- Create table
DROP TABLE IF EXISTS big_table;
```

```
CREATE TABLE big_table (
    id         bigint generated always as identity,
    payload    text,
    created_at timestamp default now()
);

-- Insert batch 1
INSERT INTO big_table (payload)
SELECT repeat(md5(random()::text), 50)
FROM generate_series(1, 700000);

SELECT pg_size_pretty(pg_total_relation_size('big_table'));

-- Insert batch 2
INSERT INTO big_table (payload)
SELECT repeat(md5(random()::text), 50)
FROM generate_series(1, 700000);

SELECT pg_size_pretty(pg_total_relation_size('big_table'));

-- Insert batch 3
INSERT INTO big_table (payload)
SELECT repeat(md5(random()::text), 50)
FROM generate_series(1, 200000);

-- Final check
SELECT pg_size_pretty(pg_total_relation_size('big_table'));
SELECT count(*) FROM big_table;
```

7.5 Phase 4 — Record timestamp and simulate disaster

Record exact timestamp before disaster

```
SELECT clock_timestamp();
```

Recorded timestamp:

```
2026-06-09 10:16:40.170546-04
```

Wait 5 seconds:

```
SELECT pg_sleep(5);
```

Simulate disaster — drop table

```
DROP TABLE big_table;
```

Verify table is gone:

```
\dt
\q
```

Force WAL archive

```
sudo -u postgres p -c "SELECT pg_switch_wal();"
```

Wait 10 seconds then verify:

```
ls -lh /var/lib/pg/16/archive/ | tail -5
```

7.6 Phase 5 — Stop Postgre

```
sudo systemctl stop postgre-16
sudo systemctl status postgre-16 --no-pager
```

7.7 Phase 6 — Preserve damaged data directory

```
mv /var/lib/pg/16/data \
   /var/lib/pg/16/data.bad_$(date +%F_%H%M%S)

ls /var/lib/pg/16/
```

7.8 Phase 7 — Restore base backup

```
mkdir -p /var/lib/pg/16/data

cp -a /pgbackup/base/base_01/* /var/lib/pg/16/data/

chown -R postgres:postgres /var/lib/pg/16/data
chmod 700 /var/lib/pg/16/data
```

Verify:

```
ls -lh /var/lib/pg/16/data/
```

7.9 Phase 8 — Configure PITR

Set recovery parameters

```
cat >> /var/lib/pg/16/data/postgre.auto.conf <<'EOF'

# ──────────────────────────────────────────
# PITR Recovery Configuration
# Target: restore to before DROP TABLE
# Recorded time: 2026-06-09 10:16:40-04
# ──────────────────────────────────────────
restore_command = 'cp /var/lib/pg/16/archive/%f %p'
recovery_target_time = '2026-06-09 10:16:43-04'
recovery_target_action = 'promote'
EOF
```

Note: We use `10:16:43` which is **3 seconds after** the recorded timestamp `10:16:40`. This ensures all committed data is recovered but the DROP TABLE is excluded.

Verify configuration:

```
tail -15 /var/lib/pg/16/data/postgre.auto.conf
```

Create recovery signal file

```
touch /var/lib/pg/16/data/recovery.signal
chown postgres:postgres /var/lib/pg/16/data/recovery.signal
ls -lh /var/lib/pg/16/data/recovery.signal
```

7.10 Phase 9 — Start recovery

```
sudo systemctl start postgre-16
```

Monitor recovery logs

```
journalctl -u postgre-16 -f
```

Expected log messages:

```
LOG:  starting point-in-time recovery to 2026-06-09 10:16:43-04
LOG:  restored log file "000000010000000200000042" from archive
LOG:  redo starts at 0/2000028
LOG:  consistent recovery state reached at 0/2000100
LOG:  recovery stopping before commit of transaction ...
LOG:  pausing at the end of recovery
LOG:  database system is ready to accept read only connections
LOG:  database promoted to a standalone database
LOG:  database system is ready to accept connections
```

7.11 Phase 10 — Verify recovery

```
sudo -u postgres p -d postgres


-- Confirm table exists
\dt

-- Count recovered rows
SELECT count(*) FROM big_table;

-- Check table size
SELECT pg_size_pretty(pg_total_relation_size('big_table'));

-- Confirm cluster is primary
SELECT pg_is_in_recovery();

-- Check current time
SELECT now();

-- Sample data
SELECT * FROM big_table LIMIT 5;
SELECT * FROM big_table ORDER BY id DESC LIMIT 5;
```

Expected results:

Check	Expected
`\dt`	`big_table` listed
`count(*)`	`1600000`
`pg_size_pretty`	`~1 GB`
`pg_is_in_recovery()`	`f`

7.12 Phase 11 — Post-recovery cleanup

```
sudo -u postgres p -d postgres -c "
ALTER SYSTEM RESET restore_command;
ALTER SYSTEM RESET recovery_target_time;
ALTER SYSTEM RESET recovery_target_action;
"

sudo systemctl reload postgre-16
```

Verify settings are cleared:

```
sudo -u postgres p -d postgres -c "
SELECT name, setting
FROM pg_settings
WHERE name IN (
    'restore_command',
    'recovery_target_time',
    'recovery_target_action'
);
"
```

---

8. Recovery Decision Matrix

Situation	Recovery Target	Command
Accidental DROP TABLE	time before drop	`recovery_target_time`
Bad deployment	named restore point	`recovery_target_name`
Exact transaction	transaction ID	`recovery_target_xid`
Latest consistent state	none needed	omit target
Specific WAL position	LSN	`recovery_target_lsn`

---

9. Troubleshooting

9.1 Common issues

Issue	Cause	Solution
WAL file not found	archive gap	check archive continuity
Recovery does not start	missing `recovery.signal`	create signal file
Wrong target time	timestamp format error	use `YYYY-MM-DD HH:MM:SS±TZ`
Permission denied	wrong ownership	`chown -R postgres:postgres /data`
Archive command fails	path wrong	verify archive directory exists
Recovery stops too early	target time too early	adjust target time forward
Recovery stops too late	target time too late	adjust target time backward

9.2 Verify WAL archive continuity

```
ls /var/lib/pg/16/archive/ | sort | head -20
ls /var/lib/pg/16/archive/ | sort | tail -20
ls /var/lib/pg/16/archive/ | wc -l
```

9.3 Check archive status

```
SELECT
    archived_count,
    failed_count,
    last_archived_wal,
    last_archived_time,
    last_failed_wal,
    last_failed_time
FROM pg_stat_archiver;
```

---

10. Best Practices

10.1 Backup

Practice	Reason
Store backups on separate storage	protect from disk failure
Compress backups after taking	save storage space
Label backups clearly	easy identification
Verify backup after creation	confirm backup integrity
Keep multiple backup generations	allow multiple recovery points

10.2 WAL archiving

Practice	Reason
Monitor archive failures	detect problems early
Set `archive_timeout`	limit data loss window
Verify archive continuity	ensure recovery is possible
Keep archive on separate storage	protect from data loss

10.3 Recovery

Practice	Reason
Always preserve damaged data dir	allow forensic analysis
Test recovery monthly	validate backup usability
Document recovery procedures	reduce recovery time
Use restore points before risky operations	simplify recovery targeting

---

11. Automation Script

11.1 Backup automation


```
#!/bin/
# ──────────────────────────────────────────────────────────
# Postgre 16 Base Backup Script
# ──────────────────────────────────────────────────────────
set -e

PGHOST="127.0.0.1"
PGPORT="5432"
PGUSER="replicator"
BACKUP_BASE="/pgbackup/base"
BACKUP_DIR="${BACKUP_BASE}/base_$(date +%F_%H%M%S)"
RETENTION_DAYS=28
LOG_FILE="/var/log/pgbackup.log"

export PGPASSWORD='ReplPass123'

echo "$(date): Starting base backup to ${BACKUP_DIR}" >> "$LOG_FILE"

pg_basebackup \
  -h "$PGHOST" \
  -p "$PGPORT" \
  -U "$PGUSER" \
  -D "$BACKUP_DIR" \
  -F p \
  -X stream \
  -P \
  -c fast \
  -l "auto_backup_$(date +%F)"

unset PGPASSWORD

echo "$(date): Backup completed: ${BACKUP_DIR}" >> "$LOG_FILE"

# Remove old backups
find "$BACKUP_BASE" -maxdepth 1 -type d -mtime +$RETENTION_DAYS -exec rm -rf
{} \;

echo "$(date): Old backups cleaned" >> "$LOG_FILE"
```

11.2 Archive monitoring script

```
#!/bin/
# ──────────────────────────────────────────────────────
# Postgre WAL Archive Monitor
# ──────────────────────────────────────────────────────

FAILED=$(sudo -u postgres p -d postgres -t -c \
  "SELECT failed_count FROM pg_stat_archiver;")

if [ "$FAILED" -gt 0 ]; then
    echo "WARNING: WAL archive failures detected: $FAILED"
    exit 1
else
    echo "OK: WAL archiving is healthy"
    exit 0
fi
```

---

12. Summary

12.1 PITR workflow summary

Step	Action	Command
1	Configure archiving	`postgre.conf`
2	Take base backup	`pg_basebackup`
3	Generate WAL	normal operations
4	Record timestamp	`SELECT clock_timestamp()`
5	Disaster occurs	`DROP TABLE`
6	Flush WAL	`SELECT pg_switch_wal()`
7	Stop Postgre	`systemctl stop`
8	Preserve data dir	`mv data data.bad`
9	Restore backup	`cp -a base_01/* data/`
10	Configure recovery	`postgre.auto.conf`
11	Create signal	`touch recovery.signal`
12	Start Postgre	`systemctl start`
13	Verify recovery	`SELECT count(*)`
14	Cleanup	`ALTER SYSTEM RESET`

12.2 Key parameters reference

```
# Archiving
wal_level             = replica
```

```
archive_mode            = on
archive_command         = 'cp %p /var/lib/pg/16/archive/%f'
archive_timeout         = 60

# Recovery
restore_command         = 'cp /var/lib/pg/16/archive/%f %p'
recovery_target_time    = 'YYYY-MM-DD HH:MM:SS±TZ'
recovery_target_action = 'promote'
```

📌 Document reference

Item	Detail
Postgre version	16
Archive directory	/var/lib/pg/16/archive
Backup directory	/pgbackup/base
Recorded disaster time	2026-06-09 10:16:40.170546-04
Recovery target time	2026-06-09 10:16:43-04
Archive count at lab start	581 WAL files

Personal details

Field	Deatils
Full name	Ahmed Sayed
Job title	Senior Technical Database Administator
Department	Database
Organization/Company	Orange Egypt
Email	Ahmedsayed.dba@gmail.com